



كلية العلوم
السملاية - مراكش
FACULTÉ DES SCIENCES
SEMLALIA - MARRAKECH

Module : Architecture des Ordinateurs

Pr. *Chakir Oumaima*, Docteur en Cybersécurité et Intelligence Artificielle

o.chakir@uca.ac.ma

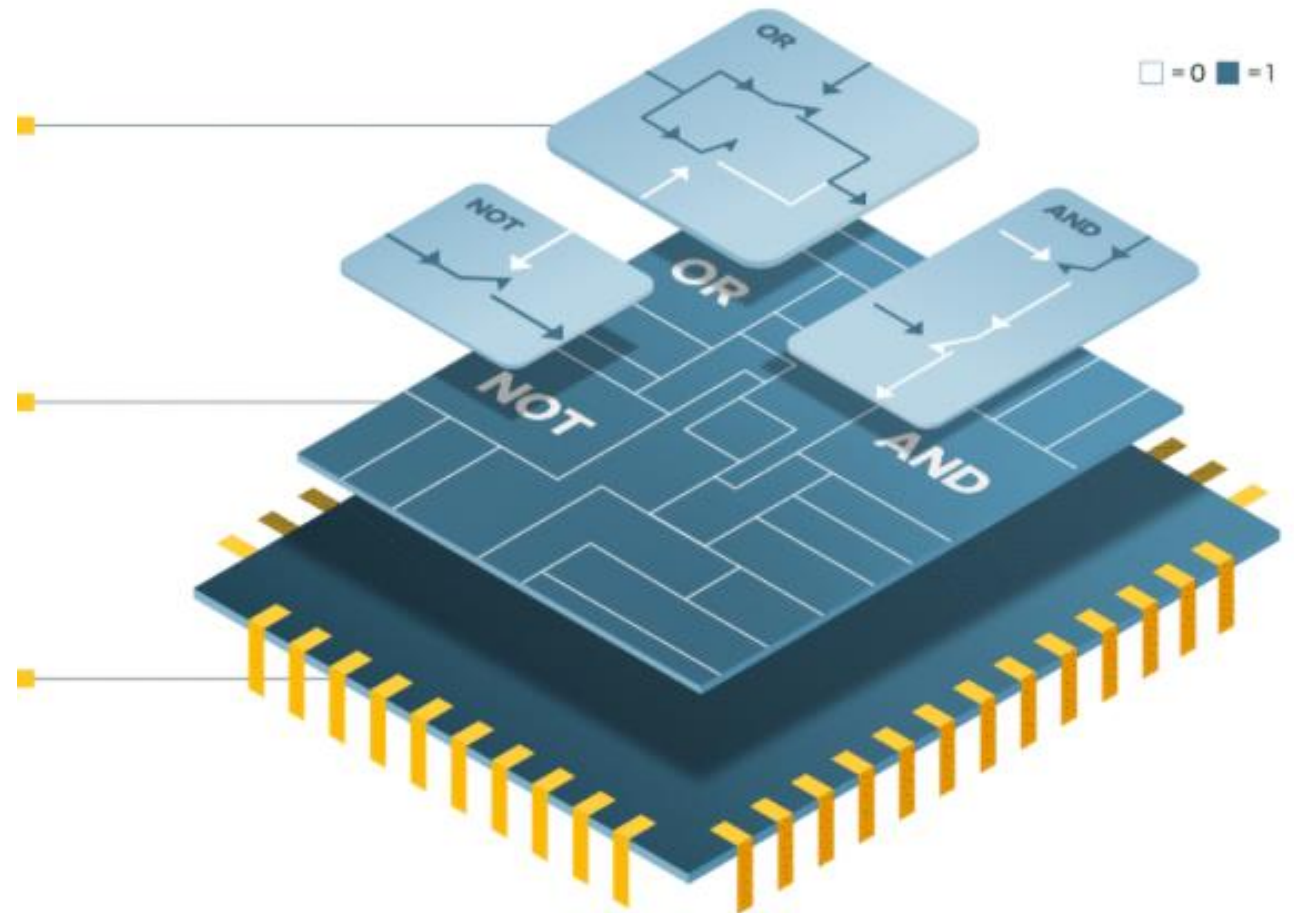
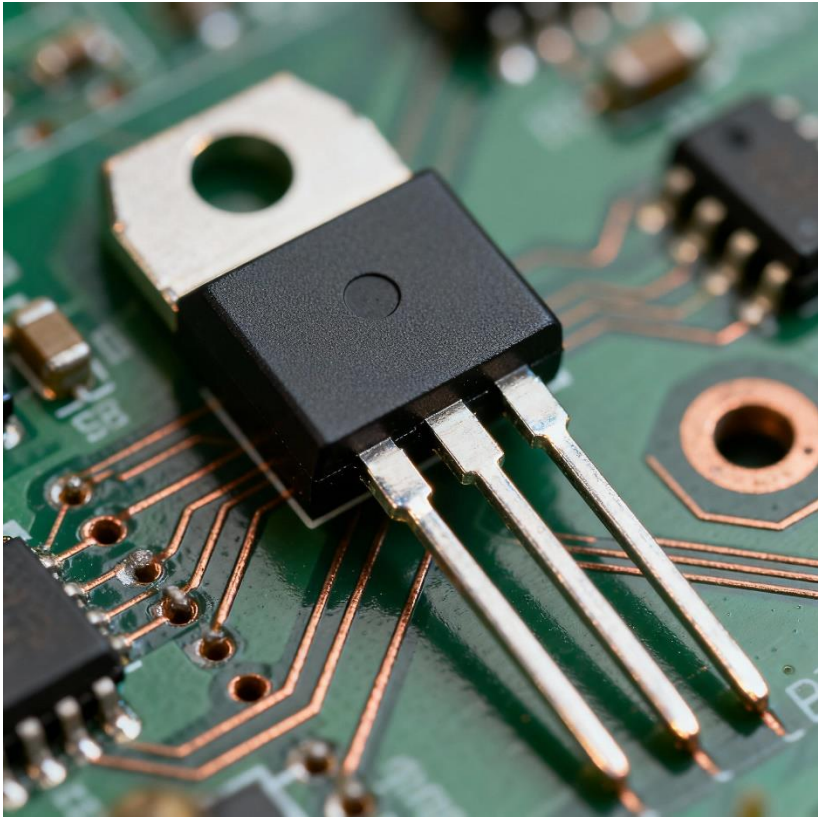
Chapitre 2: Représentation de l'information

Représentation de l'information

- ✓ Les informations traitées par un ordinateur peuvent être de différents types: texte, nombres, images, son, vidéos, etc.
- ✓ Mais elles sont toujours représentées et manipulées par l'ordinateur sous forme numérique (binaire : 0 et 1)
- ✓ Cette représentation numérique permet de les stocker en mémoire, de les traiter dans le processeur et de les envoyer à travers un réseau.

Représentation de l'information

Comment l'ordinateur représente-t-il concrètement l'information numérique ?



Représentation de l'information

- ✓ L'ordinateur *représente l'information numérique à l'aide de ses circuits électroniques*, qui utilisent des transistors pour stocker et manipuler des valeurs binaires.
- ✓ Ces circuits sont composés de *millions (voire milliards) de transistors intégrés sur des puces de silicium*.
- ✓ Un transistor est un petit composant électronique qui agit comme un interrupteur pour *laisser passer ou non le courant électrique*.

Représentation de l'information

Chaque transistor peut être dans deux états :

- ✓ **Passant (ou « fermé »)** → le courant passe → le circuit **délivre une tension entre 2 et 5 volts**
 **état logique 1**
- ✓ **Bloquant (ou « ouvert »)** → le courant ne passe pas
→ le circuit **délivre une tension entre 0 et 1 volt**
 **état logique 0**

Représentation de l'information

- ✓ En informatique, l'unité de base de la quantité d'information est le **bit (Binary Digit)**



Plus petit élément manipulable par l'ordinateur

qui ne pouvant prendre pour valeur que 0 ou 1.

- ✓ Avec **n bits**, on peut représenter : 2^n états
- ✓ **1 octet = 8 bits**, et chaque bit peut valoir 0 ou 1.

Codage de l'information

Le codage de l'information permet d'établir une correspondance qui permet sans ambiguïté de passer d'une représentation (dite externe) d'une information à une autre représentation (dite interne : sous forme binaire) de la même information, suivant un ensemble de règles précises.

Systemes de numération

- ✓ Un système de numération est un ensemble de symboles et de règles permettant de représenter les nombres.
- ✓ Il se définit par deux éléments:
 1. *La base du système*
 2. *Les symboles du système*
- ✓ En informatique, les systèmes les plus utilisés sont les suivants: Binaire, décimal, octal et hexadécimal.

Systemes de numération

La représentation d'un nombre N en base b

Soit N un nombre quelconque exprimé dans une base b . N sera noté comme suit:

$$N = (a_{n-1} a_{n-2} a_{n-3} \dots a_0)_b$$

b : base du système de numération.

a_i : symbole du système, $i = 0, \dots, n-1$. avec: $a_i < b$

Systèmes de numération

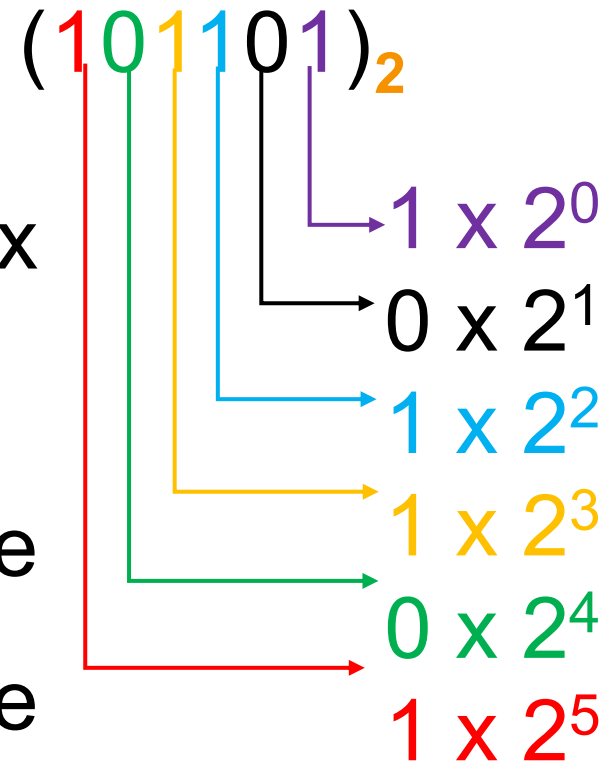
Système	Base	Symboles	Nbre de symbole
Décimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9.	10
Binaire	2	0, 1.	2
Octal	8	0, 1, 2, 3, 4, 5, 6, 7.	8
Hexadécimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.	16

Systemes de numération: Système Binaire

✓ C'est le système de numération utilisé en informatique pour la représentation des informations au niveau de l'ordinateur.

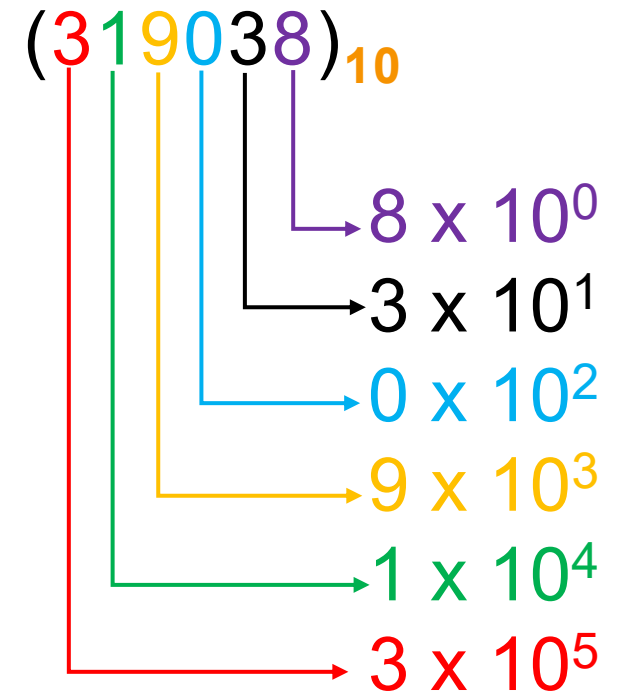
✓ Un système à base 2 ($b=2$) qui utilise deux chiffres: $\{0, 1\}$

✓ La position des 0 et des 1 indique respectivement l'absence ou la présence d'une puissance de 2.



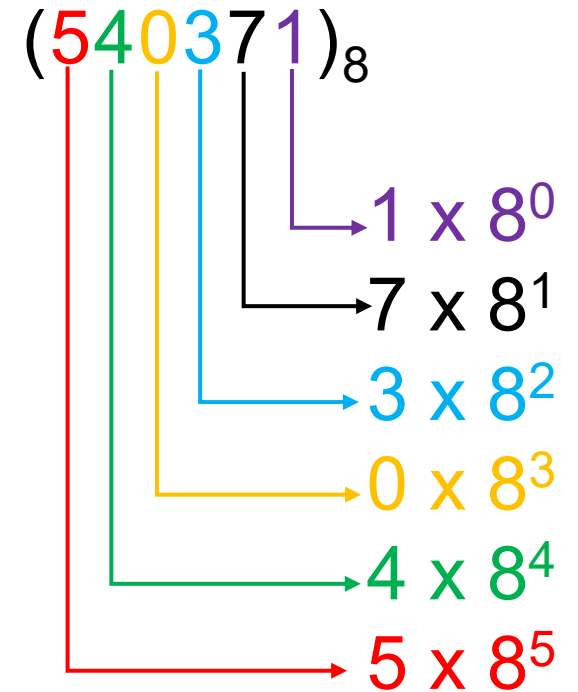
Systemes de numération: Système Décimal

- ✓ Le système de numération le plus utilisé dans la vie quotidienne.
- ✓ Il est en base 10, c'est-à-dire qu'il utilise 10 symboles : 0, 1, 2, 3, 4, 5, 6, 7, 8 et 9.



Systemes de numération: Système Octal

- ✓ La base du système octal est 8.
- ✓ En octal, les nombres sont représentés sous forme de combinaisons de chiffres parmi les suivants: 0, 1, 2, 3, 4, 5, 6, et 7.



Systemes de numération: Système Hexadécimal

Le système **hexadécimal** (**base 16**) utilise 16 chiffres pour la représentation des nombres.

Hexadécimal	Décimal
0	0
1	1
2	2
9	9
A	10
B	11
C	12
D	13
E	14
F	15

$(140FC1)_H$

- 1×16^0
- 12×16^1
- 15×16^2
- 0×16^3
- 4×16^4
- 1×16^5

Systemes de numération: Système Hexadécimal

Quiz: Indiquez, pour chaque cas, si la notation est correcte ou incorrecte. Justifiez votre réponse.

$$N_1 = (3257)_8$$

$$N_4 = (10012)_2$$

$$N_2 = (1203)_4$$

$$N_5 = (987)_{10}$$

$$N_3 = (A23F)_{16}$$

$$N_6 = (5G2)_{16}$$

Systemes de numération: Système Hexadécimal

Réponse:

$N_1 = (3257)_8 \longrightarrow$ Valide, car tous les chiffres sont compris entre 0 et 7.

$N_2 = (1203)_4 \longrightarrow$ Valide, car tous les chiffres sont compris entre 0 et 3.

$N_3 = (A23F)_{16} \longrightarrow$ Valide, car tous les chiffres sont compris entre $\{0,9\}$ et $\{A,F\}$

Systemes de numération: Système Hexadécimal

$N_4 = (10012)_2 \rightarrow$ En base 2, seuls les chiffres 0 et 1 sont permis donc la notation n'est pas correcte.

$N_5 = (987)_{10} \rightarrow$ Valide, car tous les chiffres sont compris entre 0 et 9

$N_6 = (5G2)_{16} \rightarrow$ Invalide, car en base 16, les lettres valides vont de A à F (10–15). **G** n'existe pas en hexadécimal, donc la notation est incorrecte.

Systemes de numération: Transcodage

Le transcodage ou conversion de base est l'opération qui permet de passer de la représentation d'un nombre exprimé dans une base à la représentation du même nombre mais exprimé dans une autre base.

***Conversion entre le décimal et les
autres bases binaire, octal, et Hexa***

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Soit $N = (a_{n-1} a_{n-2} a_{n-3} \dots a_0)_b$, pour avoir la représentation en décimal du nombre N exprimé dans une base b quelconque, il suffit d'effectuer le calcul suivant:

$$(N)_{10} = a_{n-1} * b^{n-1} + a_{n-2} * b^{n-2} + \dots + a_1 * b^1 + a_0 * b^0$$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

La formule générale s'écrit comme suit:

$$(N)_{10} = \sum_{i=0}^{n-1} a_i * b^i$$

Avec :

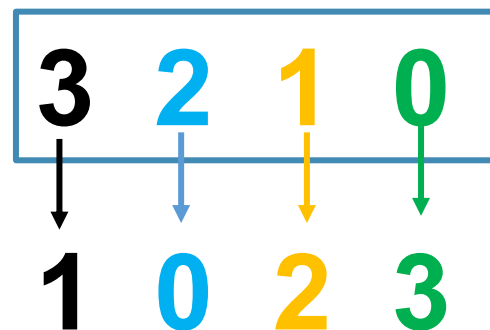
- ✓ **b :** La base
- ✓ **n :** Nombre des chiffres
- ✓ **a_i :** un chiffre de poids i (position i)

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Exemple 1 : On considère le nombre: $N = (1023)_4$

✓ **Etape 1:** Définir le poids de chaque chiffre en les numérotant de droite à gauche, en commençant par 0.



Poids des chiffres (i)

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Exemple 1 : On considère le nombre: $N = (1023)_4$

✓ **Etape 2**: Multiplier chaque chiffre a_i par la base b élevée à la puissance i : $a_i \times b^i$.

$$\begin{aligned} N &= 1 * 4^3 + 0 * 4^2 + 2 * 4^1 + 3 * 4^0 \\ &= 64 + 0 + 8 + 3 \\ &= 75 \end{aligned}$$

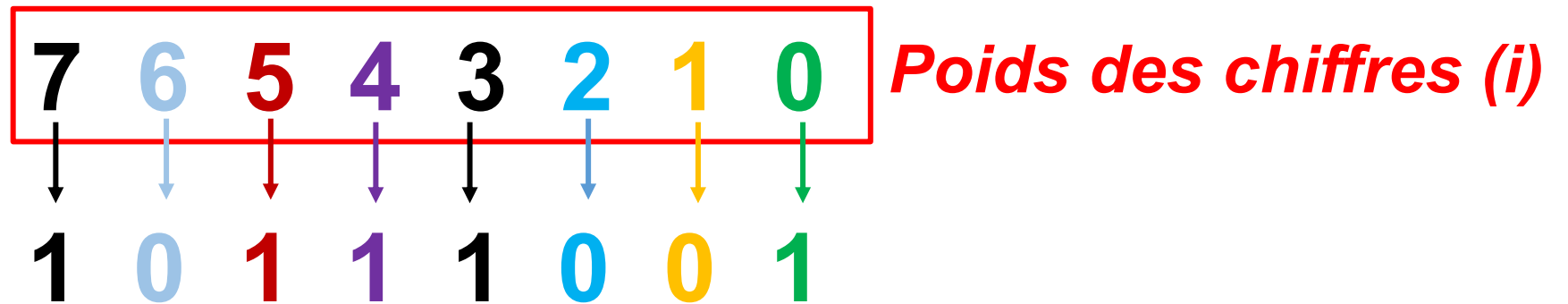
$$N = (1023)_4 = (75)_{10}$$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Exemple 2 : On considère le nombre: $N = (10111001)_2$

✓ **Etape 1:** Les poids des chiffres



Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Exemple 2 : On considère le nombre: $N = (10111001)_2$

✓ **Etape 2:** Le nombre N en décimal est calculé comme suit

$$\begin{aligned} N &= 1 * 2^7 + 0 * 2^6 + 1 * 2^5 + 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 \\ &= 128 + 0 + 32 + 16 + 8 + 0 + 0 + 1 \\ &= 185 \end{aligned}$$

$$\text{D'où: } N = (10110101)_2 = (185)_{10}$$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en une base b (2, 8, 16) vers base 10

Exemple 3 : Calculez la valeur en décimal du nombre **X1 = (175)₈** et **X2 = (A24)_H**

$$\begin{aligned}(X1)_{10} &= 1 \cdot 8^2 + 7 \cdot 8^1 + 5 \cdot 8^0 \\ &= 64 + 56 + 5 \\ &= 125\end{aligned}$$

Ainsi, **$(175)_8 = (125)_{10}$**

$$\begin{aligned}(X2)_{10} &= A \cdot 16^2 + 2 \cdot 16^1 + 4 \cdot 16^0 \\ &= 10 \cdot 16^2 + 2 \cdot 16^1 + 4 \cdot 16^0 \\ &= 2596\end{aligned}$$

Ainsi, **$(A24)_H = (2596)_{10}$**

Systemes de numération: Transcodage

*Conversion du nombre N exprimé base 10 vers une
base b (2, 8, 16)*

Il existe deux méthodes principales:

- 1. Méthode des divisions entières successives***
- 2. Méthode des soustractions successives***

Systemes de numération: Transcodage

Conversion du nombre N exprimé base 10 vers une base b (2, 8, 16)

1. Méthode des divisions entières successives : *N est itérativement divisé par « b » jusqu'à obtenir un quotient égal à 0.* La conversion du *nombre N dans la base « b » est obtenue en notant les restes de chacune des divisions* effectuées depuis la dernière division jusqu'à la première. *Les restes sont obligatoirement inférieurs à « b ».*

Exemple 1: La conversion de $(235)_{10}$ en base 2

235		2
117		1
58		0
29		1
14		0
7		1
3		1
1		1
0		0

Lire et noter les restes de bas en haut : C'est le nombre dans la nouvelle base.

Donc, $(235)_{10} = (11101011)_2$

Les restes obtenus < 2

Quotient = 0 $\rightarrow$ opération terminée

Exemple 2: La conversion de $(235)_{10}$ en base 8 et 16

✓ *Conversion de $(235)_{10}$ en base 8*

$$235 \div 8 = 29 \text{ reste } 3$$

$$29 \div 8 = 3 \text{ reste } 5$$

$$3 \div 8 = 0 \text{ reste } 3$$

Ensuite, on lit les restes de bas en haut : $(235)_{10} = (353)_8$

✓ *Conversion de $(235)_{10}$ en base 16*

$$235 \div 16 = 14 \text{ reste } 11 \rightarrow 11 \text{ en hex} = \mathbf{B}$$

$$14 \div 16 = 0 \text{ reste } 14 \rightarrow 14 \text{ en hex} = \mathbf{E}$$

On lit les restes de bas en haut : $(235)_{10} = (\mathbf{EB})_{16}$

Systèmes de numération: Transcodage

Conversion du nombre N exprimé base 10 vers une base b

2. Méthode des soustractions successives :

- a) On commence par trouver la plus grande puissance de la base qui est inférieure ou égale à N .
- b) On soustrait cette puissance du N et on répète l'opération avec le reste, en descendant jusqu'à la plus petite puissance.
- c) Le nombre de fois que chaque puissance est utilisée devient les chiffres du nombre dans la nouvelle base, du plus grand au plus petit.

Exemple 1: Convertir **$(235)_{10}$** en base 8

a) On sait que $8^0 = 1$; $8^1 = 8$; $8^2 = 64$; $8^3 = 512$.

b) $235 - 64 = 171$;

$171 - 64 = 107$;

$107 - 64 = 43$;

$43 - 8 = 35$;

$35 - 8 = 27$;

$27 - 8 = 19$;

$19 - 8 = 11$;

$11 - 8 = 3$;

$3 - 1 = 2$;

$2 - 1 = 1$;

$1 - 1 = 0$

*On a utilisé **3 fois** $8^2 \rightarrow$ le premier chiffre est **3***

*On a utilisé **5 fois** $8^1 \rightarrow$ le deuxième chiffre est **5***

*On a utilisé **3 fois** $8^0 \rightarrow$ le dernier chiffre est **3***

c). d'où **$(235)_{10} = (353)_8$**

Exemple 2: Convertir $(235)_{10}$ en base 2 et 16

Puissances de 2 ≤ 235 : 128, 64, 32, 16, 8, 4, 2, 1

- $235 - 128 = 107$
- $107 - 64 = 43$
- $43 - 32 = 11$
- $11 - 16 \rightarrow$ non, on met 0
- $11 - 8 = 3$
- $3 - 4 \rightarrow$ non, 0
- $3 - 2 = 1$
- $1 - 1 = 0$

Donc: $(235)_{10} = (11101011)_2$

Exemple 2: Convertir $(235)_{10}$ en base 2 et 16

Puissances de 16 ≤ 235 : $16^1 = 16$, $16^2 = 256$ (trop grand)

Donc on utilise $16^1 = \mathbf{16}$ et $16^0 = \mathbf{1}$

Résultat : $(235)_{10} = (\mathbf{EB})_{16}$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 8 vers 2

Exemple 1 : On considère le nombre: $N = (1023)_8$

✓ **Etape 1:** Décomposer le nombre octal en prenant chaque chiffre du nombre octal individuellement.

$(1023)_8$  1 0 2 3

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 8 vers 2

Exemple 1 : On considère le nombre: $N = (1023)_8$

✓ Etape 2: Remplacer chaque chiffre octal par sa représentation binaire sur trois bits.

1 0 2 3
001 000 010 011

Chiffre octal	Chiffre binaire équivalent
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 8 vers 2

Exemple 1: On considère le nombre: $N = (1023)_8$

- ✓ Etape 3: Regrouper les triplets de bits obtenus dans le même ordre que les chiffres du nombre octal initial pour obtenir la représentation binaire finale.

$$\begin{array}{cccc} 1 & 0 & 2 & 3 \\ 001 & 000 & 010 & 011 \end{array} \quad \longrightarrow \quad (1023)_8 = (001000010011)_2$$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 2 vers 8

Exemple 2: On considère le nombre: $N = (10111010)_2$

✓ Etape 1: En partant de la droite vers la gauche, on regroupe les chiffres binaires par paquets de trois bits

$(10111010)_2 \quad \longrightarrow \quad 010 \quad 111 \quad 010$

N.B: Si le dernier groupe (à gauche) contient moins de trois bits, on ajoute des zéros à gauche pour le compléter.

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 2 vers 8

Exemple 2: On considère le nombre: $N = (10111010)_2$

✓ Etape 2: Convertir chaque groupe de trois bits en son chiffre octal correspondant.

$$(010 \ 111 \ 010)_2 \longrightarrow (272)_8$$

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 2 vers 8

Exemple 2: Quelle est la conversion octale de $N1 = (0101011)_2$ et binaire de $N2 = (764)_8$?

✓ **Octal-Binaire:** $(0101011)_2 = (053)_8$

✓ **Binaire-Octal:** $(764)_8 = (111110100)_2$

Systemes de numération: Transcodage

Base 16 vers 2

Exemple : On considère le nombre: $N = (A2B)_{16}$

- ✓ Etape 1: Décomposer le nombre hexadécimal en prenant chaque chiffre du nombre individuellement.

$(A2B)_{16}$  **A 2 B**

Systemes de numération: Transcodage

Base 16 vers 2

Exemple : On considère le nombre: $N = (A2B)_{16}$

- ✓ Etape 2: Remplacer chaque chiffre hexadécimal par sa représentation binaire sur quatre bits

A **2** **B**
1010 **0010** **1011**

Chiffre hexadécimal	Chiffre binaire équivalent
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Systemes de numération: Transcodage

Base 16 vers 2

Exemple : On considère le nombre: $N = (A2B)_{16}$

- ✓ Etape 3: Regrouper les groupes de quatre bits obtenus dans le même ordre que les chiffres du nombre hexadécimal initial pour obtenir la représentation binaire finale.

A	2	B		
1010	0010	1011		$(A2B)_{16} = (101000101011)_2$

Systemes de numération: Transcodage

Base 2 vers 16

Exemple : On considère le nombre: $N = (10111010)_2$

- ✓ Etape 1: En partant de la droite vers la gauche, on regroupe les chiffres binaires par paquets de quatre bits.

$(10111010)_2 \quad \longrightarrow \quad 1011 \quad 1010$

Systemes de numération: Transcodage

Base 2 vers 16

Exemple : On considère le nombre: $N = (10111010)_2$

- ✓ Etape 2: Convertir chaque groupe de quatre bits en son chiffre hexadécimal correspondant.

1011 1010  (BA)_H

Systemes de numération: Transcodage

Conversion du nombre N exprimé en base 2 vers 8

Exemple 2: Quelle est la conversion hexadécimale de

$N1 = (0101011)_2$ et binaire de $N2 = (7A4)_H$?

- ✓ *Hexadécimal-Binaire: $(0101011)_2 = (2B)_H$*
- ✓ *Binaire-Hexadécimal: $(7A4)_H = (011110100100)_2$*

Systemes de numération: Transcodage

Conversion de la base i vers la base j

$$(765)_8 = (?)_H$$

$$(124)_5 = (?)_8$$

Systemes de numération: Transcodage

Conversion de la base i vers la base j

1. *Passage par la base 2* : Si les bases i et j sont des puissances de 2, la conversion s'effectue en passant par la base 2, selon le schéma suivant :

base i $\rightarrow$ base 2 $\rightarrow$ base j

2. *Passage par la base 10* : Si les bases i et j ne sont pas des puissances de 2, la conversion s'effectue en passant par la base 10, selon le schéma suivant :

base i $\rightarrow$ base 10 $\rightarrow$ base j

Systemes de numération: Transcodage

Conversion de la base i vers la base j

✓ Exemple 1: $(765)_8 = (?)_{10}$

Les bases 8 et 16 sont des puissances de 2, car $2^3 = 8$ et $2^4 = 16$ on peut donc effectuer la conversion en passant par la base 2.

a) Etape 1: Remplacer chaque chiffre octal par son équivalent en 3 bits binaires.

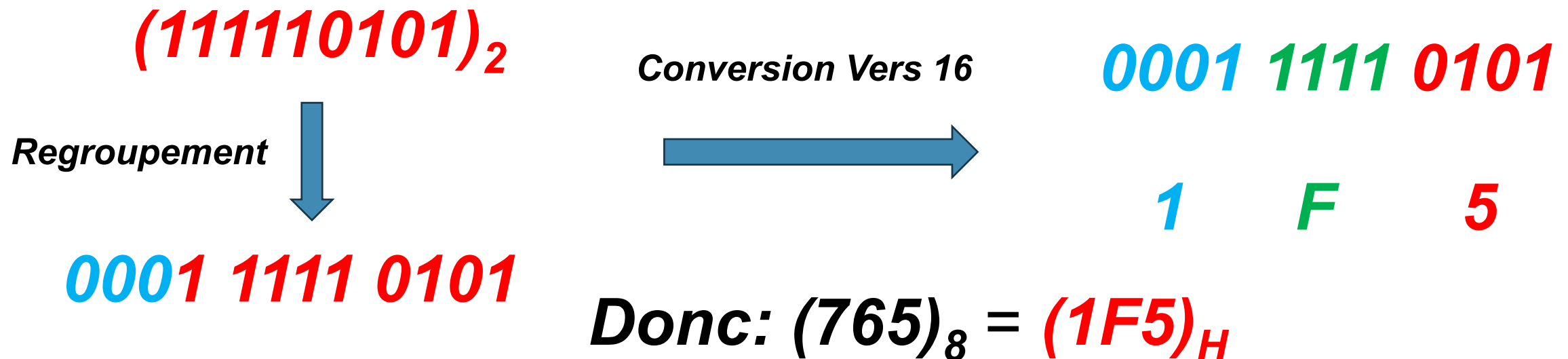
$$\begin{array}{ccc} 7 & 6 & 5 \\ 111 & 110 & 101 \end{array} \longrightarrow (765)_8 = (111110101)_2$$

Systemes de numération: Transcodage

Conversion de la base i vers la base j

✓ Exemple 1: $(765)_8 = (?)_H$

b). Étape 2 : Regrouper les bits obtenus par groupes de 4 en partant de la droite, puis convertir chaque groupe en chiffre hexadécimal.



Systemes de numération: Transcodage

Conversion de la base i vers la base j

✓ Exemple 2: $(132)_5 = (?)_7$

Les bases 5 et 7 ne sont pas des puissances de 2, donc on doit effectuer la conversion en passant par la base 10.

a) Étape 1 : Base 5 $\rightarrow$ Base 10

$$(132)_5 = 1 * 5^2 + 3 * 5^1 + 2 * 5^0 = (42)_{10}$$

Systèmes de numération: Transcodage

Conversion de la base i vers la base j

✓ Exemple 2: $(132)_5 = (?)_7$

b) Étape 2 : Base 10 $\rightarrow$ Base 7

En utilisant la méthode des divisions ou des soustractions successives, on obtient : $(41)_{10} = (60)_7$

Donc: $(132)_5 = (60)_7$

Codage des entiers relatifs

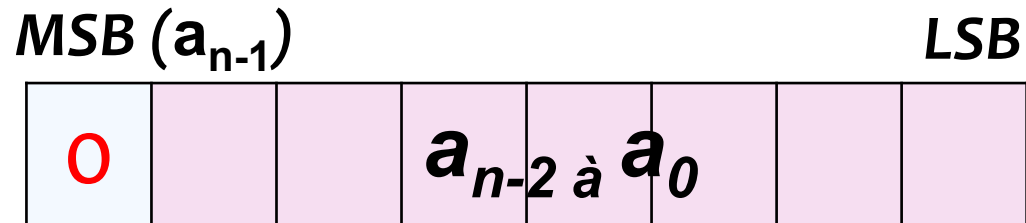
Codage des entiers relatifs

- ✓ Un entier relatif est un nombre sans virgule, qui peut être positif ou négatif.
- ✓ Les principaux modes de codage des entiers relatifs:
 - a) ***Codage en signe et valeur absolue (ou "binaire signé")***
 - b) ***Codage en complément à 1 (ou "Complément Logique ou Restreint (CL/CR) ")***
 - c) ***Codage en complément à 2 (ou" Complément Vrai (CV)"***

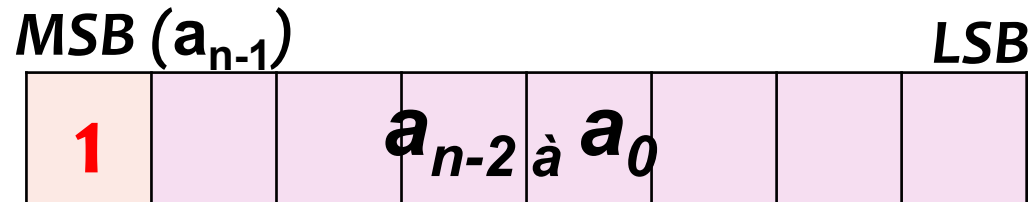
Codage des entiers relatifs

a) Codage en signe et valeur absolue (ou "binaire signé")

Dans un mot binaire de n bits, le bit de poids fort (MSB : *Most Significant Bit*) sert à indiquer le signe du nombre : il vaut 0 pour un nombre positif et 1 pour un nombre négatif, tandis que les $n - 1$ bits restants représentent la valeur absolue du nombre.



Nombre Positif



Nombre Négatif

Codage des entiers relatifs

a) Codage en signe et valeur absolue (ou "binaire signé")

Exemple: En codage binaire signé, le nombre $N = (-13)_{10}$ est représenté sur 8 bits comme suit :

a) Étape 1 : Écrire la valeur absolue en binaire

$$(13)_{10} \xrightarrow{\text{Conversion binaire}} (00001101)_2$$

b) Étape 2 : Ajouter le bit de signe

Le nombre est négatif donc on remplace le bit de signe (MSB) par 1.

1	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

$$\text{Donc: } (-13)_{10} = (10001101)_2$$

Codage des entiers relatifs

a) Codage en signe et valeur absolue (ou "binaire signé")

1. Capacité de représentation:

Si le codage se fait sur n bit, on peut représenter:

- ✓ Les valeurs positives de 0 à $2^{(n-1)} - 1$,
- ✓ Et autant de valeurs négatives de -0 à $-(2^{(n-1)} - 1)$

Exemple: Avec 8 bits les valeurs possibles allant de -127 à 127:

- ✓ Les valeurs positives: 0 à 127
- ✓ Les valeurs négatives: -0 à -127

Codage des entiers relatifs

a) Codage en signe et valeur absolue (ou "binaire signé")

2. Limitations du binaire signé :

- ✓ Deux représentations inutiles possibles du zéro : **+0** et **-0**
(Sur 8 bits: 00000000 et 10000000 respectivement).
- ✓ Opérations arithmétiques plus lentes et plus complexes.

Codage des entiers relatifs

b). Codage en complément à 1

- ✓ Le bit du poids fort est toujours utilisé pour le signe
- ✓ Un nombre positif est représenté normalement en binaire, tandis qu'un **nombre négatif** est obtenu en **inversant tous les bits** (en remplaçant les 0 par des 1 et les 1 par des 0) représentant la valeur absolue du nombre.

Codage des entiers relatifs

b). Codage en complément à 1

Exemple: En codage CL, le nombre $N = (-13)_{10}$ est représenté sur 8 bits comme suit :

a) Étape 1 : Écrire la valeur positive en binaire

$(13)_{10}$ Conversion binaire $\longrightarrow$ $(00001101)_2$

b) Étape 2 : Inverser tous les bits de la valeur positive pour obtenir le nombre négatif.



Codage des entiers relatifs

b). Codage en complément à 1

1. Capacité de représentation:

Si le codage se fait sur **n bit**, donc les valeurs possibles vont

de : $-(2^{(n-1)} - 1)$ à $2^{(n-1)} - 1$

2. Limitations du binaire signé :

- ✓ Deux représentations inutiles du zéro.
- ✓ Les opérations arithmétiques sont plus simples que dans le codage binaire signé, mais ce codage présente encore certaines limites.

Codage des entiers relatifs

c) Codage en complément à 2 (ou "Complément Vrai (CV)")

- ✓ Le type plus utilisé car il permet de *réaliser facilement les opérations arithmétiques* sans traitement particulier du signe.
- ✓ Pour un *nombre positif*, on écrit simplement sa représentation binaire normale.
- ✓ Pour *un nombre négatif*, on ajoute simplement à 1 à son complément à 1.

Codage des entiers relatifs

c) **Codage en complément à 2 (ou "Complément Vrai (CV)")**

Exemple: En codage CV, le nombre $N = (-13)_{10}$ est représenté sur

8 bits comme suit :

a) Étape 1 : Écrire la valeur positive en binaire

$$(13)_{10} \xrightarrow{\text{Conversion binaire}} (00001101)_2$$

b) Étape 2 : Inverser tous les bits de la valeur positive pour obtenir son complément à 1



Codage des entiers relatifs

c) **Codage en complément à 2 (ou "Complément Vrai (CV)")**

Exemple: En codage CV, le nombre $N = (-13)_{10}$ est représenté sur

8 bits comme suit :

C) Étape 3: Pour obtenir la représentation binaire du nombre négatif, on ajoute la valeur 1.

CL: 11110010

+ 00000001

11110011

Donc: $(-13)_{10} = (11110011)_2$

Codage des entiers relatifs

c) *Codage en complément à 2 (ou "Complément Vrai (CV)")*

- ✓ Une seule représentation du zéro (00000000).
- ✓ Les opérations arithmétiques sont simples et naturelles
- ✓ Codage unique et cohérent
- ✓ Si le codage se fait sur n bit, donc les valeurs possibles vont de :

$$-(2^{(n-1)}) \quad \text{à} \quad 2^{(n-1)} - 1$$

Codage des nombres réels

Codage des nombres réels

- ✓ Les nombres réels sont ceux qui peuvent contenir une partie entière et une partie décimale.
- ✓ Pour les **nombres réels**, il existe deux principaux codages :
 - a) La virgule fixe (Fixed-point)*
 - b) La virgule flottante**

Codage des *nombre*s réels

1. La virgule fixe (Fixed-point)

- ✓ La position de la virgule est fixée à un endroit précis pour tous les nombres.
- ✓ La machine sait exactement où commence la partie fractionnaire et où finit la partie entière.
- ✓ Virgule fixe: $d_{n-1} d_{n-2} d_{n-3} \dots d_2 d_1 d_0$, $d_{-1} d_{-2} \dots d_{-m}$ avec n et m fixe.


Partie Entière *Partie Fractionnaire*

Codage des nombres réels

a) La virgule fixe (Fixed-point): b vers 10

On considère $N = (d_{n-1} \dots d_1 d_0, d_{-1} d_{-2} \dots d_{-m})_b$ représenter en base 10 comme suit:

$$(N)_{10} = d_{n-1} * b^{n-1} + \dots + d_1 * b^1 + d_0 * b^0 + d_{-1} * b^{-1} + d_{-2} * b^{-2} + \dots + d_{-m} * b^{-m}$$

Avec:

d_{n-1} : Digit de poids fort

d_{-m} : Digit de poids faible

n, m : fixe

Codage des *nombre*s réels

a) La virgule fixe (Fixed-point): b vers 10

Exemple 1: On considère $N = (10,1)_b$ un nombre qui s'exprime en base B par 3 bits avec $n=2$ et $m=1$:

✓ En binaire: $(10,1)_2 = 1*2^1 + 0*2^0 + 1*2^{-1} = (2,5)_{10}$

✓ En octale: $(10,1)_8 = 1*8^1 + 0*8^0 + 1*8^{-1} = (8,125)_{10}$

✓ En hexadécimal: $(10,1)_{16} = 1*16^1 + 0*16^0 + 1*16^{-1} = (16,0625)_{10}$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): 10 vers b

On considère $N = (77.75)_{10}$ un nombre que l'on veut exprimer en base B.

Etape 1: Convertir la partie entière en base B

✓ ***Cas 1: En base $B=2$ avec $n=8$ et $m=2$:***

En utilisant la division entière ou soustraction successive, on trouve:

$$(77)_{10} \xrightarrow{\text{Conversion binaire sur } n \text{ bits}} (01001101)_2$$

Donc la représentation de la partie entière en base 2 est:

$$(01001101)_2$$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): 10 vers b

On considère $N = (77.75)_{10}$ un nombre que l'on veut exprimer en base B.

Etape 2: Convertir la partie fractionnaire en base B

✓ ***Cas 1: En base $B=2$ avec $n=8$ et $m=2$:***

On multiplie successive la partie fractionnaire par B et on garde la partie entière, comme suit:

$0.75 * 2 = 1.5$ // le premier chiffre de la partie fractionnaire est 1

$0.5 * 2 = 1.0$ // le deuxième chiffre de la partie fractionnaire est 1

Donc la représentation de la partie fractionnaire en base 2 avec $m=2$:
 $(0.11)_2$

Codage des nombres réels

b) La virgule fixe (Fixed-point): 10 vers b

On considère $N = (77.75)_{10}$ un nombre que l'on veut exprimer en base B.

Etape 3: Combinaison des deux parties

✓ ***Cas 1: En base $B=2$ avec $n=8$ et $m=2$:***

$$(77.75)_{10} \xrightarrow{\text{Conversion binaire}} (01001101.11)_2$$

Avec:

01001101 : Partie entière ($n=8$)

0.11 : Partie fractionnaire ($m=2$)

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): 10 vers b

On considère $N = (77.75)_{10}$ un nombre que l'on veut exprimer en base B .

✓ Cas 2: En base $B=8$

Etape 1: Convertir la partie entière en base B

$$(77)_{10} \xrightarrow{\text{Conversion octale}} (115)_8$$

Etape 2: Convertir la partie fractionnaire en base B

$$(0.75)_{10} \xrightarrow{* 8} (6)_8$$

$$\text{Donc: } (77.75)_{10} = (115.6)_8$$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): 10 vers b

On considère $N = (77.75)_{10}$ un nombre que l'on veut exprimer en base B.

✓ Cas 3: En base $B=16$

Etape 1: Convertir la partie entière en base B

$$(77)_{10} \xrightarrow{\text{Conversion hexadécimale}} (4D)_{16}$$

Etape 2: Convertir la partie fractionnaire en base B

$$(0.75)_{10} \xrightarrow{* 16} (C)_{16}$$

$$\text{Donc: } (77.75)_{10} = (4D.C)_{16}$$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): Vers 2

On considère $N = (77.75)_B$ un nombre que l'on veut exprimer en base 2.

✓ Cas 1: En base $B=16$ avec $n=m=8$

Etape 1: Convertir la partie entière en base B

$$(77)_{16} \xrightarrow{\text{Conversion binaire sur 4 bits}} (01110111)_2$$

Etape 2: Convertir la partie fractionnaire en base B

$$(0.75)_{16} \xrightarrow{\text{Conversion binaire sur 4 bits}} (01110101)_2$$

$$\text{Donc: } (77.75)_{16} = (01110111.01110101)_2$$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): Vers 2

On considère $N = (77.75)_B$ un nombre que l'on veut exprimer en base 2.

✓ Cas 2: En base $B=8$ avec $n=m=8$

Etape 1: Convertir la partie entière en base B

$$(77)_8 \xrightarrow{\text{Conversion binaire sur 3 bits}} (\mathbf{00}111111)_2$$

Etape 2: Convertir la partie fractionnaire en base B

$$(0.75)_8 \xrightarrow{\text{Conversion binaire sur 3 bits}} (111101\mathbf{00})_2$$

$$\text{Donc: } (77.75)_8 = (\mathbf{00}111111.11110100)_2$$

Codage des *nombre*s réels

b) La virgule fixe (Fixed-point): Vers 2

À retenir :

Cas	Ce qu'on fait
Si la partie entière a moins de bits que n	On ajoute des 0 à gauche
Si la partie fractionnaire a moins de bits que m	On ajoute des 0 à droite
Si elle dépasse n ou m	On tronque ou arrondit

Codage des *nombre*s réels

c) La virgule fixe: Les principaux challenges

- ✓ Plage de représentation limitée: Comme la virgule est fixée, la valeur maximale et la valeur minimale qu'on peut représenter sont très limitées.
- ✓ Précision fixe et limitée: Le nombre de bits de la partie fractionnaire est fixe. Donc, la précision est identique pour tous les nombres, même pour les très grands ou très petits.

Codage des *nombre*s réels

2.1). La virgule flottante

Un nombre est représenté en virgule flottante dans la base X

s'il est mis sous la forme : $\mathbf{N} = \pm \mathbf{M} * \mathbf{B}^E$

où :

- ✓ $M \rightarrow$ **mantisse** $\in [1, B[$;
- ✓ $B \rightarrow$ **base** (souvent 2 en binaire)
- ✓ $E \rightarrow$ **exposant**, qui déplace la virgule vers la gauche ou la droite

La position de la virgule "flotte" selon la valeur de l'exposant.

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

- ✓ La norme IEEE 754 a été créée pour garantir la **fiabilité**, l'**exactitude** et la **cohérence** des calculs en virgule flottante sur tous les systèmes informatiques.
- ✓ Elle définit une *représentation universelle des nombre*s réels sous la forme :

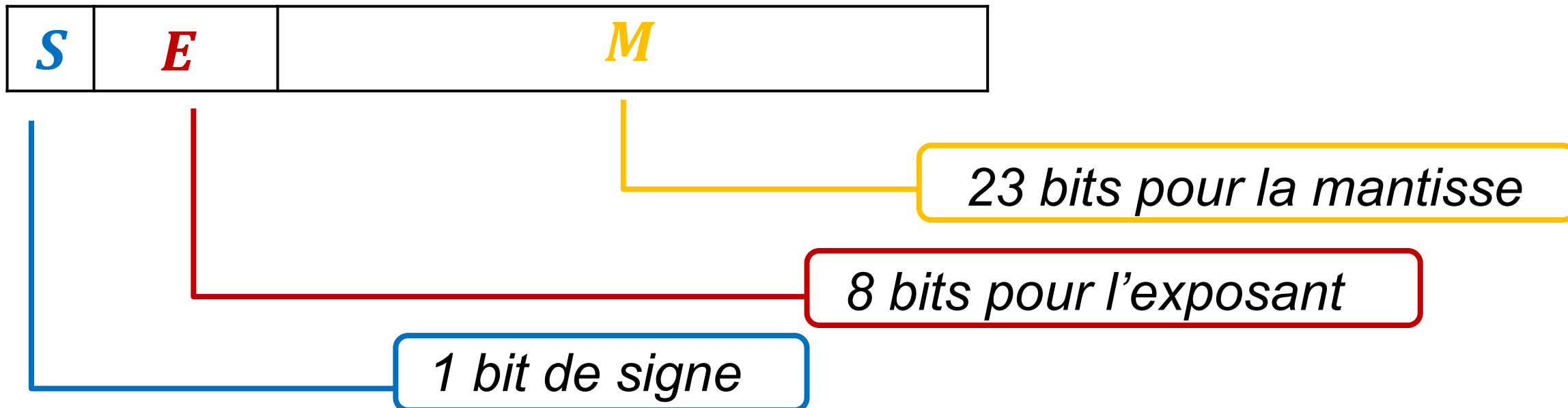
$$N = \pm 1.M * B^E$$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

Elle définit deux formats pour représenter des nombres à virgule flottante:

1. Simple précision: Le nombre réel Codé sur 32 bits

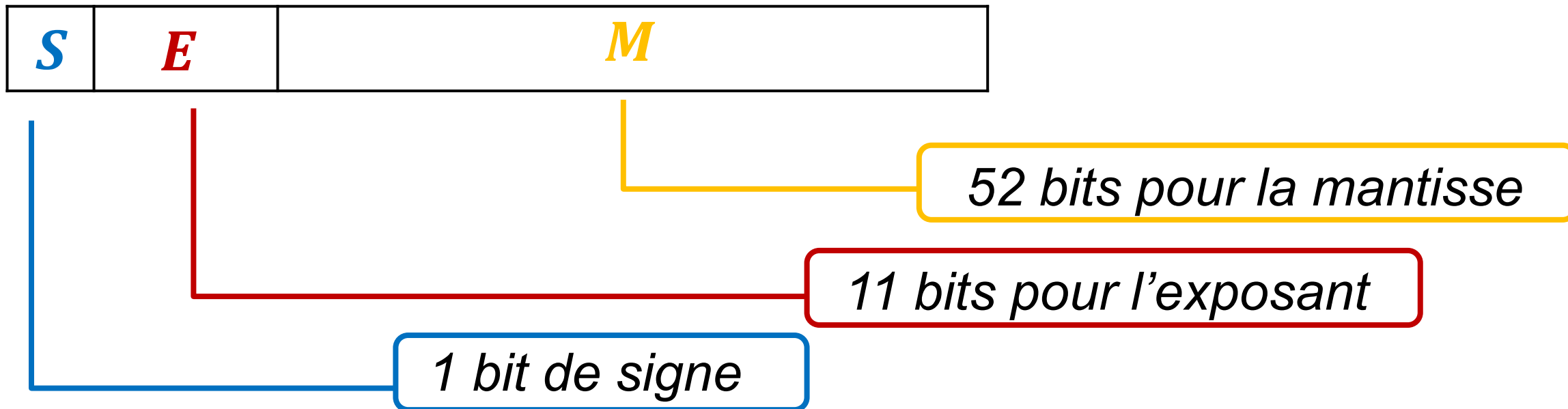


Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

Elle définit deux formats pour représenter des nombres à virgule flottante dans une suite binaire:

2. Double précision: Le nombre réel codé sur 64 bits



Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 1: Convertir la valeur absolue en binaire

Après avoir converti la partie entière et la partie fractionnaire, on obtient :

$$(9,5)_{10} \xrightarrow{\text{Conversion Binaire}} (1001.1)_2$$

Avec:

1001: Partie entière

0.1: Partie fractionnaire

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 2: Normaliser le nombre (Valeur Absolue)

Déplacez la virgule binaire pour obtenir **un seul chiffre significatif non nul (le "1") à gauche de la virgule.**

$$(1001.1)_2 \xrightarrow{\text{Normalisation}} 1.\underbrace{0011}_{\text{Mantisse}} * \underbrace{2^3}_{\text{Exposant}}$$

The diagram illustrates the normalization of the binary number $(1001.1)_2$. A red arrow labeled "Normalisation" points to the normalized form $1.0011 * 2^3$. The mantissa "0011" is highlighted in blue and labeled "Mantisse" with a bracket. The base "2" is highlighted in orange and labeled "Base" with a bracket, and the exponent "3" is highlighted in green and labeled "Exposant" with a bracket.

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

a) Etape 3.1: Coder le bit de signe (0 : positif, 1 : négatif)

Le nombre N est *négatif*. Par conséquent, *le bit de signe (le premier bit) est 1.*

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

b) Etape 3.2: Coder l'exposant selon le format

✓ Cas 1: En utilisant Simple précision

On commence par ajouter un biais constant (127 en simple précision) à l'exposant réel, comme suit:

$$\begin{aligned}\text{Exposant biaisé} &= \text{Exposant réel} + 127 \\ &= 3 + 127 = 130\end{aligned}$$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

b) Etape 3.2: Coder l'exposant selon le format

✓ Cas 1: En utilisant Simple précision

Ensuite en code l'exposant biaisé sur 8 bits et on obtient :

10000010_2

Donc le codage de l'exposant en simple précision est:

$(10000010)_2$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

b) Etape 3.2: Coder l'exposant selon le format

✓ Cas 2: En utilisant Double précision

$$\begin{aligned}\text{Exposant biaisé} &= \text{Exposant réel} + 1023 \\ &= 3 + 1023 = 1026\end{aligned}$$

Ensuite en code l'exposant biaisé sur 11 bits, on obtient :

$$(10000000010)_2$$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

c) Etape 3.3: Coder Mantisse selon le format

✓ Cas 1: En utilisant Simple précision

En simple précision, **la mantisse est codée sur 23 bits** donc elle doit être **complétée par des zéros** à droite:

$(0011)_2$ Conversion Binaire sur 23 bits $\rightarrow (00110000000000000000000)_2$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

On considère $N = (-9,5)_{10}$ que l'on veut le code en base 2 selon **IEEE 754**

Etape 3: Encodage des composants

c) Etape 3.3: Coder Mantisse selon le format

✓ Cas 2: En utilisant double précision

En double précision, **la mantisse est codée sur 52 bits** donc elle doit être **complétée par des zéros** à droite:

$(0011)_2$ Conversion Binaire sur 52 bits $\longrightarrow$ $(001100000000.....00000000)_2$

Codage des *nombre*s réels

2.2). La virgule flottante: Norme IEEE 754

Caractéristiques	Simple Précision	Double Précision
Signe	1 bit	1 bit
Exposant	8 bits	11 bits
Biais de l'exposant	127	1023
Variation de l'exposant	-126 à +127	-1022 à +1023
Mantisse	23 bits	52 bits
Taille totale	32 bits	64 bits
Taille totale	4 octets	8 octets

Codage des *nombre*s réels

Notions de précision, d'arrondi et d'erreurs de codage

- ✓ La **précision** d'un nombre représente le nombre de chiffres significatifs qu'on peut stocker et utiliser correctement dans un calcul.
- ✓ Plus la mantisse contient de bits (23 en simple précision, 52 en double précision), plus on peut représenter de chiffres binaires exacts après le point.

Codage des *nombre*s réels

Notions de précision, d'arrondi et d'erreurs de codage

- ✓ L'**arrondi** est la modification d'un nombre pour l'adapter au nombre de chiffres que la machine peut stocker.
- ✓ Une **erreur de codage** se produit lorsque le nombre stocké dans l'ordinateur diffère du nombre réel que l'on veut représenter.

Codage des Caractères

Codage des caractères

- ✓ En informatique, chaque caractère (ex. : A, b, ?, é, 3, etc.) est représenté par un nombre.
- ✓ Ce nombre est ensuite converti en binaire pour être stocké ou transmis par l'ordinateur.
- ✓ Les principaux systèmes de codage des caractères:
 - 1. ASCII (American Standard Code for Information Interchange)***
 - 2. Unicode***

Codage des caractères

1. ASCII (American Standard Code for Information Interchange)

1.1 ASCII Standard

- ✓ Le code ASCII a été défini dans les années 1960 pour standardiser la représentation des caractères dans les ordinateurs.
- ✓ Chaque caractère (lettre, chiffre, symbole, commande) est représenté par un nombre entier compris entre 0 et 127.
- ✓ Il utilise 7 bits pour représenter les caractères, mais l'ordinateur ajoute un bit zéro supplémentaire à gauche pour remplir l'octet.

Control Characters				Graphic Symbols											
Name	Dec	Binary	Hex	Symbol	Dec	Binary	Hex	Symbol	Dec	Binary	Hex	Symbol	Dec	Binary	Hex
NUL	0	0000000	00	space	32	0100000	20	@	64	1000000	40	'	96	1100000	60
SOH	1	0000001	01	!	33	0100001	21	A	65	1000001	41	a	97	1100001	61
STX	2	0000010	02	"	34	0100010	22	B	66	1000010	42	b	98	1100010	62
ETX	3	0000011	03	#	35	0100011	23	C	67	1000011	43	c	99	1100011	63
EOT	4	0000100	04	\$	36	0100100	24	D	68	1000100	44	d	100	1100100	64
ENQ	5	0000101	05	%	37	0100101	25	E	69	1000101	45	e	101	1100101	65
ACK	6	0000110	06	&	38	0100110	26	F	70	1000110	46	f	102	1100110	66
BEL	7	0000111	07	'	39	0100111	27	G	71	1000111	47	g	103	1100111	67
BS	8	0001000	08	(	40	0101000	28	H	72	1001000	48	h	104	1101000	68
HT	9	0001001	09	)	41	0101001	29	I	73	1001001	49	i	105	1101001	69
LF	10	0001010	0A	*	42	0101010	2A	J	74	1001010	4A	j	106	1101010	6A
VT	11	0001011	0B	+	43	0101011	2B	K	75	1001011	4B	k	107	1101011	6B
FF	12	0001100	0C	,	44	0101100	2C	L	76	1001100	4C	l	108	1101100	6C
CR	13	0001101	0D	-	45	0101101	2D	M	77	1001101	4D	m	109	1101101	6D
SO	14	0001110	0E	.	46	0101110	2E	N	78	1001110	4E	n	110	1101110	6E
SI	15	0001111	0F	/	47	0101111	2F	O	79	1001111	4F	o	111	1101111	6F
DLE	16	0010000	10	0	48	0110000	30	P	80	1010000	50	p	112	1110000	70
DC1	17	0010001	11	1	49	0110001	31	Q	81	1010001	51	q	113	1110001	71
DC2	18	0010010	12	2	50	0110010	32	R	82	1010010	52	r	114	1110010	72
DC3	19	0010011	13	3	51	0110011	33	S	83	1010011	53	s	115	1110011	73
DC4	20	0010100	14	4	52	0110100	34	T	84	1010100	54	t	116	1110100	74
NAK	21	0010101	15	5	53	0110101	35	U	85	1010101	55	u	117	1110101	75
SYN	22	0010110	16	6	54	0110110	36	V	86	1010110	56	v	118	1110110	76
ETB	23	0010111	17	7	55	0110111	37	W	87	1010111	57	w	119	1110111	77
CAN	24	0011000	18	8	56	0111000	38	X	88	1011000	58	x	120	1111000	78
EM	25	0011001	19	9	57	0111001	39	Y	89	1011001	59	y	121	1111001	79
SUB	26	0011010	1A	:	58	0111010	3A	Z	90	1011010	5A	z	122	1111010	7A
ESC	27	0011011	1B	;	59	0111011	3B	[	91	1011011	5B	{	123	1111011	7B
FS	28	0011100	1C	<	60	0111100	3C	\	92	1011100	5C		124	1111100	7C
GS	29	0011101	1D	=	61	0111101	3D	]	93	1011101	5D	}	125	1111101	7D
RS	30	0011110	1E	>	62	0111110	3E	^	94	1011110	5E	~	126	1111110	7E
US	31	0011111	1F	?	63	0111111	3F	_	95	1011111	5F	Del	127	1111111	7F

Codage des caractères

1. ASCII (American Standard Code for Information Interchange)

1.2 ASCII étendu

- ✓ Avec seulement 128 caractères, ASCII standard est insuffisant pour représenter les caractères accentués, les symboles spéciaux ou les caractères d'autres langues.
- ✓ Au lieu de 7 bits, ASCII étendu utilise 8 bits par caractère:

$2^8=256$ caractères possibles

- ✓ Les 128 premiers codes restent identiques à l'ASCII standard.
- ✓ Les 128 suivants sont utilisés pour ajouter de nouveaux caractères.

Codage des caractères

1. ASCII (American Standard Code for Information Interchange)

1.2 ASCII étendu

- a) Même avec 256 caractères, ce n'était **toujours pas suffisant** pour :
 - ✓ **Représenter toutes les langues du monde** (arabe, chinois, japonais...etc),
 - ✓ Ni tous **les symboles modernes** (mathématiques, emoji, etc.).
- b) Chaque région avait développé **son propre code local**, ce qui provoquait:
 - ✓ Des **incompatibilités entre systèmes** (un même code affichait des symboles différents) ;
 - ✓ Des **problèmes d'échange de données** entre ordinateurs de pays différents

Codage des caractères

2. UNICODE: Codage Moderne

✓ Un standard international créé en 1991 pour représenter tous les caractères de toutes les langues dans un système unique et cohérent.

✓ L'idée de base du code UNICODE:

“Everyone in the world should be able to use their own language on phones and computers.”

<https://home.unicode.org/>

Codage des caractères

2. UNICODE: Codage Moderne

Chaque caractère dans Unicode reçoit un numéro unique, noté en hexadécimal sous la forme :

U+XXXX où **XXXX** est le code du caractère

Caractère	Code Unicode
0	U+FF10
👉	U+270C
ﻝ	U+0644

Codage des caractères

2. UNICODE: Codage Moderne

Unicode définit un ensemble de caractères universel, mais pour les **stocker ou les transmettre**, on utilise des **formats d'encodage** appelés **UTF-Unicode Transformation Format**:

- ✓ **UTF-8**

- ✓ **UTF-16**

- ✓ **UTF-32**

Codage des caractères

<i>Type</i>	<i>Nombre de caractères</i>	<i>Langues couvertes</i>
ASCII Standard	128	Anglais uniquement
ASCII étendu	256	Langues européennes
Unicode	+1 000 000	Toutes les langues

Opérations Arithmétiques Binaires

Opérations arithmétiques en binaire

Addition binaire

L'addition binaire est l'addition de nombres en base 2, qui suit des règles similaires à l'addition décimale mais avec les chiffres 0 et 1 uniquement.

<i>Bit A</i>	<i>Bit B</i>	<i>Résultat</i>	<i>Retenue</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Opérations arithmétiques en binaire

Addition binaire

Exemple 1 : Calculer $A+B$ avec, $A=B= (1101)_2$

N.B: On additionne colonne par colonne en partant de la droite et on ajoute la retenue finale à gauche.

$$\begin{array}{r} \textcolor{red}{1} \quad \textcolor{red}{1} \quad \textcolor{red}{1} \\ + \quad 1011 \\ \quad 1011 \\ \hline \textcolor{red}{1} 0110 \end{array} \quad \longrightarrow \quad (1011)_2 + (1011)_2 = (10110)_2$$

Opérations arithmétiques en binaire

Addition binaire

Exemple 2 : Calculer $A+B$ avec, $A= (1011)_2$ et $B= (1101)_2$

$$\begin{array}{r} \textcolor{red}{1} \textcolor{red}{1} \textcolor{red}{1} \textcolor{red}{1} \\ + \quad 1011 \\ \quad 1101 \\ \hline \textcolor{red}{1} 1000 \end{array} \quad \longrightarrow \quad (1011)_2 + (1101)_2 = (11000)_2$$

Opérations arithmétiques en binaire

Addition Binaire

Exemple 3 : Calculer $A+B$ avec, $A= (101)_2$ et $B= (1101)_2$

N.B: Les nombres binaires peuvent être de longueurs différentes; on complète souvent avec des ***zéros à gauche pour aligner les colonnes.***

$$\begin{array}{r}
 1 1 1 \\
 0101 \\
 1101 \\
 \hline
 10010
 \end{array}$$

Opérations arithmétiques en binaire

Multiplication Binaire

<i>Bit A</i>	<i>Bit B</i>	<i>Résultat</i>
0	0	0
1	0	0
0	1	0
1	1	1

Opérations arithmétiques en binaire

Multiplication Binaire

Exemple 1: Calculer $A*B$ avec, $A = (1010)_2$ et $B = (1001)_2$

$$\begin{array}{r} 1010 \\ * 1001 \\ \hline 1010 \\ 0000. \\ 0000.. \\ 1010... \\ \hline 1011010 \end{array}$$

→ $(1010)_2 * (1001)_2 = (1011010)_2$

Opérations arithmétiques en binaire

Multiplication Binaire

Exemple 2: Calculer $A*B$ avec, $A = (111)_2$ et $B = (101)_2$

$$\begin{array}{r} 111 \\ * 101 \\ \hline 111 \\ 000. \\ 111. \\ \hline 100011 \end{array}$$

→ $(111)_2 * (101)_2 = (100011)_2$

Opérations arithmétiques en binaire

Soustraction Binaire

La soustraction binaire se fait **bit par bit**, en commençant **par la droite** (le bit de poids faible), comme en décimal. On utilise les règles suivantes :

<i>A-B</i>	<i>Résultat</i>	Emprunt
0-0	0	0
1-0	1	0
1-1	0	0
0-1	1	1

Opérations arithmétiques en binaire

Soustraction Binaire

Cas 1: Classique

Exemple 1: Calculer $A-B$ avec, $A= (1010)_2$ et $B= (1001)_2$

$$\begin{array}{r} \overset{\text{0}}{\cancel{1}} \overset{\text{10}}{\cancel{0}} \\ - 1001 \\ \hline 0001 \end{array}$$

$$(1010)_2 - (1001)_2 = (0001)_2$$

Règle d'emprunt

- ✓ Si le bit courant est **0** et qu'on doit soustraire **1**, on doit **emprunter 1** au bit immédiatement à gauche.
- ✓ Mais en binaire, emprunter **1** équivaut à **ajouter 2** (car en base 2).
- ✓ Le bit de gauche diminue alors de 1.
- ✓ Si ce bit de gauche est lui-même **0**, il faut **continuer à emprunter** jusqu'à trouver un **1**.

Opérations arithmétiques en binaire

Soustraction Binaire

Cas 2: Avec Complément à 2

Exemple 2: Calculer $A-B$ avec, $A = (1010)_2$ et $B = (1001)_2$

Etape 1: Calculer le complément à 2 de B

$$\begin{aligned}\text{Complément à 2} &= \text{Complément à 1} + 1 \\ &= 0110 + 1\end{aligned}$$

$$\text{Donc: } \text{Cà2} = (0111)_2$$

Opérations arithmétiques en binaire

Soustraction Binaire

Cas 2: Avec Complément à 2

Exemple 1: Calculer $A-B$ avec, $A = (1010)_2$ et $B = (1001)_2$

Etape 2: Ajouter A au complément à 2 de B

$$A-B = A + C_2(B)$$

$$\begin{array}{r} 1010 \\ + 0111 \\ \hline \end{array}$$

$$\begin{array}{r} 1010 \\ + 0111 \\ \hline 10001 \end{array}$$

N.B. : Il faut ignorer le dépassement de bit (s'il y en a) et conserver le résultat. Ainsi, le résultat final est :

$$(1010)_2 + (0111)_2 = (0001)_2$$

Opérations arithmétiques en binaire

Soustraction Binaire

Cas 2: Avec Complément à 2

Exemple 2: Calculer $A-B$ avec, $A = (1101)_2$ et $B = (1010)_2$

$$A-B = A + C\grave{a}2(B) \quad \text{Avec } C\grave{a}2(B) = (0110)_2$$

$$\begin{aligned} A-B &= 1101 + 0110 \\ &= 10011 \end{aligned}$$

Comme on travaille sur 4 bits, on ignore le dépassement du 5^e bit.

Donc le résultat final est: $(0011)_2$

Opérations arithmétiques en binaire

Division Binaire

Exemple: Supposons A/B donc:

- ✓ Si la division possible $\longrightarrow$ *Quotient = 1*
- ✓ Si la division impossible $\longrightarrow$ *Quotient = 0*

Opérations arithmétiques en binaire

Division Binaire

Exemple 1: Calculer A/B avec, $A = (1010)_2$ et $B = (10)_2$

$$\begin{array}{r} 1010 \quad 10 \\ 10 \overline{) 01} \\ \underline{00} \\ 10 \end{array}$$

1. Sélectionnez les bits du dividende (A) de gauche à droite jusqu'à former un nombre **supérieur ou égal au diviseur (B)**.
2. **Mettre le quotient correspondant** : si le nombre obtenu $\geq$ diviseur $\rightarrow 1$, sinon $\rightarrow 0$.
3. **Soustrayez le diviseur de ce nombre**, puis descendez le bit suivant du dividende et répétez le processus jusqu'à ce que tous les bits aient été traités.

Donc, $(1010)_2 / (10)_2 = (101)_2$

Opérations arithmétiques en binaire

Division Binaire

Exemple 1: Calculer A/B avec, $A = (1010)_2$ et $B = (101)_2$

$$\begin{array}{r|l} 1010 & 101 \\ 101 \downarrow & 10 \\ \hline 000 & \end{array}$$

→ $(1010)_2 / (101)_2 = (10)_2$

Représentation des instructions machine

Représentation des instructions machine

Notion d'instruction machine

- ✓ Une instruction machine est ***un ordre donné au processeur pour lui indiquer quelle opération exécuter.***
- ✓ Chaque instruction permet au processeur de ***réaliser un traitement simple:***
 - Effectuer une addition,
 - Lire une donnée en mémoire,
 - Sauter vers une autre instruction, etc.

Représentation des instructions machine

Notion d'instruction machine

La représentation des instructions machine est *un codage binaire normalisé où chaque instruction est divisée en champs correspondant à l'opération et ses opérandes*, selon *une architecture spécifique* (par exemple Intel, ARM, etc).

Représentation des instructions machine

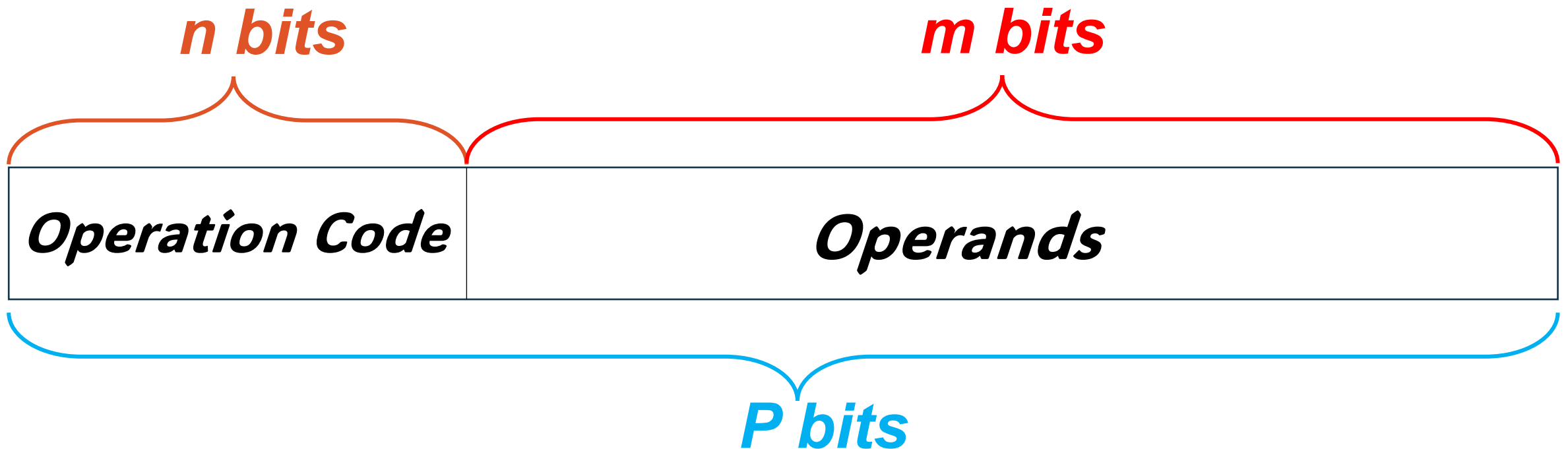
En résumé :

- ✓ Une instruction machine = un **ordre** pour le processeur.
- ✓ Sa représentation = **le code binaire** qui correspond à cet ordre.

Représentation des *instructions machine*

Structure générale d'une instruction

Une *instruction machine* est généralement composée de *deux champs*, disposés de manière *binaire en mémoire*.



Représentation des instructions machine

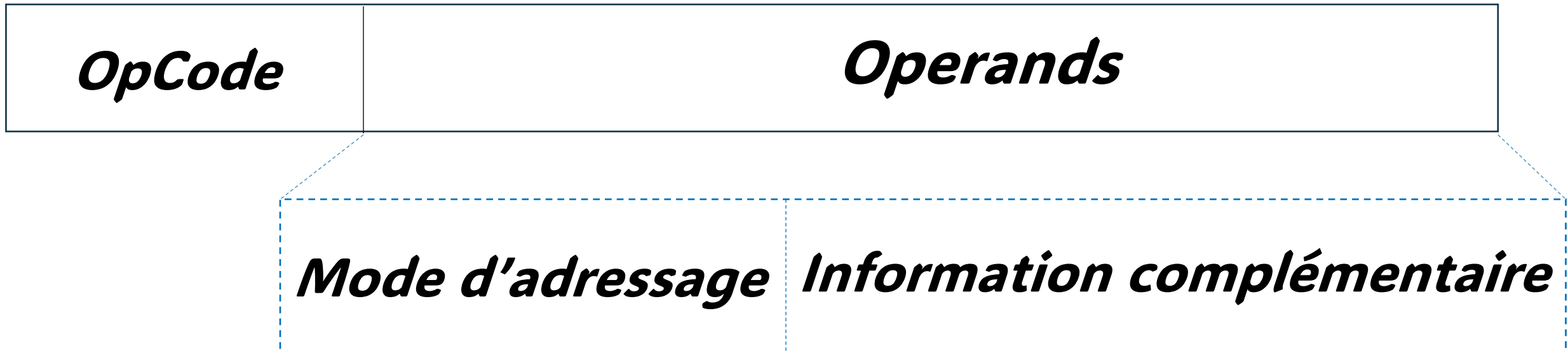
Structure générale d'une instruction

1) Code opération

- ✓ Indique au processeur l'opération à effectuer.
- ✓ Chaque opération correspond à un code binaire bien précis.
- ✓ Le nombre total de codes possibles dépend du nombre de bits
m: 2^n instructions différentes
- ✓ L'ensemble de ces opérations forme le jeu d'instructions du processeur.

2) Opérandes

- ✓ Cette partie de l'instruction spécifie sur **quelles données l'opération doit agir.**
- ✓ En réalité, le champ opérande est souvent divisé en deux sous-champs :



2) Opérandes

2.1) Le champ « Modes d'adressage »

Ils indiquent comment le processeur peut identifier l'emplacement des opérandes nécessaires à l'exécution d'une instruction :

- a) **Immédiat** : La valeur de l'opérande est directement incluse dans l'instruction.
- b) **Direct** : L'instruction contient l'adresse exacte de l'opérande en mémoire.
- c) **Indirect** : L'instruction contient l'adresse d'un registre ou d'une case mémoire qui elle-même contient l'adresse réelle de l'opérande.
- d) **Registre** : L'instruction indique directement quel registre contient l'opérande, ce qui signifie que la donnée est déjà présente dans le CPU.

2) Opérandes

2.2) Le champ « Information Complémentaire »

- ✓ Le processeur lit le mode d'adressage pour savoir **comment interpréter le contenu du champ information complémentaire.**
- ✓ Fournir **les détails nécessaires pour que le processeur puisse accéder à l'opérande.**
- ✓ Il peut contenir :
 - a) une **valeur immédiate**,
 - b) un **numéro de registre**,
 - c) une **adresse mémoire**,
 - d) une **adresse d'une autre adresse** (dans le cas du mode indirect).

Types des instructions machine

1. Instructions de transfert

- ✓ Permettent de déplacer des données entre les registres, la mémoire ou les périphériques sans les modifier.
- ✓ Assurent la circulation des données au sein du système.

2. Instructions arithmétiques

- ✓ Réalisent des opérations mathématiques sur les opérandes.
- ✓ Utilisées pour effectuer des calculs tels que l'addition, la soustraction, la multiplication ou la division.

Types des instructions machine

3. Instructions logiques

- ✓ Effectuent des opérations logiques (AND, OU, XOR, CMP, NOT) sur les bits.
- ✓ Elles ne font pas de calculs comme les instructions arithmétiques, mais elles servent à tester, combiner ou modifier des valeurs binaires.

Types des instructions machine

4. Instructions de saut ou de branchement

- ✓ Permettent de ***rompre l'exécution séquentielle des instructions*** d'un programme.
- ✓ ***L'instruction exécutée à la suite d'un saut*** n'est pas celle qui suit immédiatement l'instruction de saut, mais ***celle dont l'adresse a été spécifiée dans l'instruction de saut.***
- ✓ On distingue deux types d'instructions de sauts: Saut conditionnel et inconditionnel

Types des instructions machine

4. Instructions de saut ou de branchement

4.1) Saut Inconditionnel

- ✓ Une instruction de saut inconditionnel *interrompt l'exécution normale* du programme et *redirige le processeur vers une autre adresse, sans vérifier de condition.*
- ✓ Par exemple, **JMP** (Jump) transfère toujours le contrôle à l'adresse cible du programme.

Types des instructions machine

4. Instructions de saut ou de branchement

4.2) Saut Conditionnel

- ✓ Une instruction de saut conditionnel permet de **modifier le flux d'exécution du programme seulement si une condition est remplie.**
- ✓ Cette condition est vérifiée grâce aux flags dans le registre d'état de l'UAL, comme zéro, négatif, overflow ou retenue.
- ✓ Par exemple, **JZ 150** signifie saute à l'adresse 150 **si le résultat précédent était nul**. Si le résultat n'est pas nul, le processeur ne saute pas et continue l'exécution normale.